

Rapport de veille technologique

API Kanban Board — William Rauwens-Oliver — Septembre 2026

Contexte

En tant que responsable technique, je dois choisir la technologie backend d'un nouveau projet, avec cinq semaines pour former l'équipe. Trois technologies étaient envisagées : NestJS, Symfony et Spring Boot. Pour les évaluer, j'ai développé une API de Kanban Board (gestion d'utilisateurs, de listes et de cartes), un projet simple mais complet permettant de juger rapidement les forces et faiblesses de chaque solution.

Présentation des technologies

NestJS est un framework backend Node.js écrit en TypeScript, inspiré de l'architecture d'Angular (modules, contrôleurs, services). *Avantages* : structure claire et modulaire, typage fort, injection de dépendances, large écosystème JavaScript, courbe d'apprentissage rapide pour qui connaît JS/TS. *Inconvénients* : performances brutes inférieures aux langages compilés, écosystème plus jeune que ses concurrents, dépendance à Node.js pour les tâches lourdes.

Symfony est un framework PHP mature et robuste, très utilisé pour les applications web d'entreprise. *Avantages* : écosystème riche et stable, excellente documentation, composants réutilisables, forte communauté francophone, idéal pour des applications web complètes. *Inconvénients* : verbeux, courbe d'apprentissage plus raide, PHP parfois perçu comme moins moderne pour les API pures.

Spring Boot est un framework Java de référence pour les applications d'entreprise. *Avantages* : extrêmement performant et fiable, adapté aux gros systèmes, écosystème très complet, standard dans les grandes entreprises. *Inconvénients* : configuration lourde, consommation mémoire élevée, verbosité de Java, prise en main longue.

Comparaison selon les critères techniques

Runtime / RAM. NestJS (Node.js) est léger et rapide à démarrer, avec une faible empreinte mémoire, mais moins performant sur les calculs intensifs. Spring Boot (JVM) consomme davantage de RAM au démarrage mais offre d'excellentes performances en charge. Symfony (PHP) se situe entre les deux.

Scalabilité. Les trois technologies sont scalables, mais l'architecture non bloquante de NestJS facilite le passage à l'échelle horizontale ; Symfony s'appuie sur des workers PHP-FPM et Spring Boot sur un modèle multi-thread.

Portabilité. Node.js (NestJS) et la JVM (Spring Boot) sont multiplateformes et adaptés aux conteneurs ; PHP (Symfony) l'est également.

Maturité. Symfony (2005) et Spring Boot (Spring, 2003) sont très matures ; NestJS (2017), plus récent, bénéficie d'une adoption rapide et solide.

Maintenabilité. Le typage fort de TypeScript et l'architecture modulaire de NestJS rendent le code clair et évolutif ; Symfony s'appuie sur des conventions strictes et Spring Boot sur un typage robuste mais verbeux.

Design patterns. Les trois reposent sur des patterns éprouvés : injection de dépendances et MVC. NestJS y ajoute ses modules, décorateurs et DTO, Symfony ses services et son Event Dispatcher, Spring Boot l'IoC et l'AOP.

Justification du choix : NestJS plutôt que Symfony et Spring Boot

J'ai choisi NestJS en le comparant à ses concurrents. Face à Spring Boot (Java), NestJS est bien plus léger et rapide à mettre en place : Spring Boot est puissant, mais sa configuration est lourde, sa consommation mémoire élevée et sa prise en main longue, ce qui est disproportionné pour une API à développer en cinq semaines avec une équipe à former.

Face à Symfony (PHP), NestJS repose sur TypeScript, offrant un typage fort et une détection des erreurs dès l'écriture. Symfony excelle pour des applications web classiques rendant leurs pages, mais NestJS est plus naturel pour une API REST pure destinée à un frontend séparé ou une application mobile.

Enfin, sur le plan personnel, j'utilise quotidiennement PHP dans mon travail : ce projet était l'occasion idéale de sortir de cet environnement et d'ajouter une nouvelle corde à mon arc en montant en compétence sur une technologie moderne. Choisir NestJS plutôt que de rester sur Symfony m'a permis de découvrir TypeScript et une architecture pensée API-first.

Son architecture modulaire et son écosystème (JWT, Prisma, Swagger) ont permis de construire une API structurée et évolutive efficacement. NestJS offrait donc le meilleur compromis entre montée en compétence, modernité et clarté.

Conclusion

Après comparaison sur les critères de runtime, scalabilité, portabilité, maturité, maintenabilité et design patterns, NestJS s'est imposé comme le choix le plus adapté à ce projet : moderne, structuré, rapide à prendre en main et parfaitement taillé pour une API REST, tout en gardant un code maintenable et évolutif.